

LangChain으로 RAG 시작하기

GENAI DAY4 RAG 프로젝트 노트북 완성본

Document loaders · Text splitters · Embeddings · Vector stores · Retrievers · Question answering

이 PDF는 완성된 Jupyter Notebook의 Markdown 설명, 코드 셀, 기존 실행 출력 및 최종 TODO 구현 셀을 읽기 쉬운 문서 형식으로 정리한 것입니다.

실행 전제조건: Colab 또는 Jupyter 환경에서 패키지를 설치하고, OPENAI_API_KEY와 Demian.pdf를 준비해야 합니다.

Cell 000

last modified date : 2026.03.15

제작 : 박광석 (모두의연구소)

Cell 001

랭체인으로 RAG 시작하기

해당 노트는 Langchain으로 RAG를 구현하기 위해 필요한

각 컴포넌트인 Document Loaders, Text splitters, Text embeddings, Vectorstores, Retriever를 다룹니다

Cell 002

Step 0 : 설치와 준비

Langchain 설치 및 Gemini API 키를 등록하도록 합니다.

Cell 003

```
from google.colab import drive
drive.mount('/content/drive')
```

Captured output

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive",
force_remount=True).
```

Cell 004

```
!pip install -U -q langchain langchain-openai
!pip install -U -q langchain-community langchain-core
!pip install -U langchain-text-splitters
```

Captured output

```
[Large captured output omitted for readability; the original notebook cell is retained.]
```

Cell 005

```
import os
```

Cell 006

```
from google.colab import userdata
os.environ['OPENAI_API_KEY'] = userdata.get('OPENAI_API_KEY')
```

Cell 007

```
#! curl ipinfo.io
```

Cell 008

```
from langchain_openai import ChatOpenAI

# OpenAI API
# "gpt-4o", "gpt-4-turbo", "gpt-3.5-turbo"
llm = ChatOpenAI(
    model="gpt-4o",
    temperature=0.0,
)
```

Cell 009

```
!pip install -q pypdf pdf2image docx2txt pdfminer unstructured #
```

Cell 010

Step 1 : Document Loaders 사용해보기

Cell 011

Document Loader는 다양한 형태의 원본 데이터를 LLM이 이해할 수 있는 Document 객체(text + metadata)로 변환하는 역할을 합니다.

PDF, 웹페이지, CSV와 같이 형식이 서로 다른 문서들을 일관된 구조로 파싱하여, 이후 Chunking-Embedding-검색(Retrieval) 단계에서 바로 사용할 수 있도록 만들어줍니다.

즉, Document Loader는 RAG 파이프라인의 가장 첫 단계에서 "데이터를 읽을 수 있는 형태로 정리하는 역할을 담당합니다.

공식 문서에서는 지원되는 다양한 Loader 목록을 확인할 수 있습니다.
https://python.langchain.com/docs/modules/data_connection/document_loaders/

Cell 012

PDFLoader 사용

이번 실습에서는 가장 많이 사용되는 문서 형식인 PDF 파일을 대상으로 PyPDFLoader를 사용해 문서를 불러옵니다.

실습을 위해, 질의응답에 활용하고 싶은 PDF 파일을 먼저 Colab 환경(또는 Drive)에 업로드해주세요.

PDFLoader는 각 페이지를 하나의 Document 단위로 변환하며, 이 단계에서 생성된 문서들은 이후 Text Splitter를 통해 의미 단위로 다시 분할됩니다.

Cell 013

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("/content/Demian.pdf")
pages = loader.load_and_split()
```

Cell 014

```
pages[0]
```

Captured output

```
Document(metadata={'producer': 'Adobe Acrobat Standard DC 19 Paper Capture Plug-in', 'creator': 'ScanFix(TM) Enhanced', 'creationdate': '2015-09-10T01:40:29+00:00', 'moddate': '2019-01-30T17:47:47+01:00', 'source': '/content/Demian.pdf', 'total_pages': 182, 'page': 0, 'page_label': '1'}, page_content='DEMIAN \n• \nDownloaded from https://www.holybooks.com')
```

Cell 015

```
print(pages[10])
```

Captured output

```
page_content='TWO WOR.LDS
Finally, out of sheer nervousness, I began to talk. I
invented a long story of robbery, in which I featured as
the hero. One night in the corner by the mill a friend
and I had stolen a whole sackful of apples, not just
ordinary apples but pippins, golden pippins of the best
kind at that. I was taking refuge in my story from the
dangers of the moment and found no difficulty in invent-
ing and relating it. In order not to dry up too soon and
perhaps become involved in something worse, I gave full
rein to my narrative powers. One of us, I reported, had
always stood guard while the other sat in the tree and
chucked the apples down, and the sack had got so heavy
that in the end we had to open it and leave half behind,
but we came back half an hour later and fetched them
too.
I hoped for some applause at the end of my story; I
had warmed up to the narrative at last, carried away by
my own eloquence. The two smaller boys were silent,
waiting, but Franz Kromer gave me a penetrating look
through his narrow, blue eyes. "Is that yarn true?" he
asked in a menacing tone.
"Yes," I said.
"Really and truly?"
"Yes, really and truly," I asserted defiantly while I
choked inwardly with fear.
"Can you swear to it?"
I was very afraid but I said 'Yes' without hesitation.
"Hand on your heart?"
"Hand on my heart,"
"Right then," he said and turned away.
I thought this was all very satisfactory and I was glad
15
Downloaded from https://www.holybooks.com' metadata={'producer': 'Adobe Acrobat Standard DC 19 Paper Capture
Plug-in', 'creator': 'ScanFix(TM) Enhanced', 'creationdate': '2015-09-10T01:40:29+00:00', 'moddate':
'2019-01-30T17:47:47+01:00', 'source': '/content/Demian.pdf', 'total_pages': 182, 'page': 10, 'page_label':
'11'}
```

Cell 016

출력 결과를 보기 쉽게 확인하기 위해,
Document 객체 전체가 아닌 실제 텍스트 본문이 담긴 page_content만 선택하여 확인해보겠습니다.

Cell 017

```
print(pages[10].page_content)
```

Captured output

TWO WOR.LDS

Finally, out of sheer nervousness, I began to talk. I invented a long story of robbery, in which I featured as the hero. One night in the corner by the mill a friend and I had stolen a whole sackful of apples, not just ordinary apples but pippins, golden pippins of the best kind at that. I was taking refuge in my story from the dangers of the moment and found no difficulty in inventing and relating it. In order not to dry up too soon and perhaps become involved in something worse, I gave full rein to my narrative powers. One of us, I reported, had always stood guard while the other sat in the tree and chucked the apples down, and the sack had got so heavy that in the end we had to open it and leave half behind, but we came back half an hour later and fetched them too.

I hoped for some applause at the end of my story; I had warmed up to the narrative at last, carried away by my own eloquence. The two smaller boys were silent, waiting, but Franz Kromer gave me a penetrating look through his narrow, blue eyes. "Is that yarn true?" he asked in a menacing tone.

"Yes," I said.

"Really and truly?"

"Yes, really and truly," I asserted defiantly while I choked inwardly with fear.

"Can you swear to it?"

I was very afraid but I said 'Yes' without hesitation.

"Hand on your heart?"

"Hand on my heart,"

"Right then," he said and turned away.

I thought this was all very satisfactory and I was glad

15

Downloaded from <https://www.holybooks.com>

Cell 018

CSVLoader

CSV 파일은 행(row) 단위로 구조화된 데이터를 담고 있는 형식으로, LangChain의 CSVLoader를 사용하면 각 행을 하나의 Document 객체로 변환할 수 있습니다.

이렇게 변환된 문서들은 이후 PDF나 웹 문서와 동일하게 Embedding, VectorStore, Retrieval 단계에서 함께 활용할 수 있습니다.

실습을 위해, CSV 파일을 먼저 Colab 환경(또는 Drive)에 업로드해주세요.

Cell 019

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader("/content/titanic.csv")

data = loader.load()
```

Cell 020

```
data[:3]
```

Captured output

```
[Document(metadata={'source': '/content/titanic.csv', 'row': 0}, page_content='PassengerId: 1\nSurvived: 0\nPclass: 3\nName: Braund, Mr. Owen Harris\nSex: male\nAge: 22\nSibSp: 1\nParch: 0\nTicket: A/5 21171\nFare: 7.25\nCabin: \nEmbarked: S'), Document(metadata={'source': '/content/titanic.csv', 'row': 1}, page_content='PassengerId: 2\nSurvived: 1\nPclass: 1\nName: Cumings, Mrs. John Bradley (Florence Briggs Thayer)\nSex: female\nAge: 38\nSibSp: 1\nParch: 0\nTicket: PC 17599\nFare: 71.2833\nCabin: C85\nEmbarked: C'), Document(metadata={'source': '/content/titanic.csv', 'row': 2}, page_content='PassengerId: 3\nSurvived: 1\nPclass: 3\nName: Heikkinen, Miss. Laina\nSex: female\nAge: 26\nSibSp: 0\nParch: 0\nTicket: STON/O2. 3101282\nFare: 7.925\nCabin: \nEmbarked: S')]
```

Cell 021

웹베이스로더

웹베이스 로더는 웹페이지에 포함된 텍스트 콘텐츠를 직접 파싱하여 Document 객체로 변환하는 역할을 합니다. 이를 통해 뉴스 기사, 블로그 글, 공지사항과 같은 실시간으로 업데이트되는 웹 문서를 RAG 시스템의 지식 소스로 활용할 수 있습니다.

이번 실습에서는 실제 뉴스 기사를 예제로 사용하여, 웹페이지의 내용을 불러오고 텍스트 형태로 변환하는 과정을 살펴봅니다.

실습에 사용할 웹페이지는 다음과 같습니다.

<https://it.chosun.com/news/articleView.html?idxno=2023092111831>

Cell 022

```
from langchain_community.document_loaders import WebBaseLoader
```

Captured output

```
WARNING:langchain_community.utils.user_agent:USER_AGENT environment variable not set, consider setting it to identify your requests.
```

Cell 023

```
loader = WebBaseLoader("https://it.chosun.com/news/articleView.html?idxno=2023092111831")
documents = loader.load()

#print(documents[0].page_content)
```

Cell 024

주석을 해제하고 코드를 실행하면, 해당 웹페이지에 포함된 본문 텍스트 전체를 불러와 확인할 수 있습니다.

웹페이지, PDF, CSV 등 서로 다른 형식의 문서들이 모두 텍스트 형태로 정상적으로 파싱된 것을 확인할 수 있습니다.

이제 이 텍스트를 전처리(불필요한 요소 제거, 정제) 한 뒤, Chunking과 Embedding 단계에 활용할 수 있습니다.

Cell 025

Step2 : TextSplitters 사용해보기

Text Splitter는 긴 텍스트 문서를 의미를 유지한 작은 단위(Chunk) 로 분할하는 역할을 합니다.

LLM은 한 번에 처리할 수 있는 토큰 수에 제한이 있기 때문에, 문서를 그대로 입력하는 대신 Splitter를 통해 분할된 여러 Chunk를 입력받아 처리하게 됩니다.

이 과정을 통해 긴 문서에서도 토큰 길이 제약을 극복하고, 필요한 부분만 효율적으로 검색할 수 있습니다.

분할된 각 Chunk는 이후 단계에서 1:1로 Embedding되어 VectorStore에 저장되며, 이 Chunk 단위가 RAG 시스템에서 검색과 응답의 기본 단위가 됩니다.

Cell 026

```
from langchain_text_splitters import CharacterTextSplitter, RecursiveCharacterTextSplitter
```

Cell 027

CharacterTextSplitter는

하나의 고정된 구분자(separator)를 기준으로 텍스트를 분할하는 방식입니다.

구현이 단순하고 직관적이지만,

문서 구조에 따라 분할된 Chunk가 토큰 제한을 초과하는 경우가 발생할 수 있습니다.

반면, RecursiveCharacterTextSplitter는
줄바꿈, 문장 구분자, 구두점 등 여러 구분자를 순차적으로 적용하며
텍스트를 재귀적으로 분할합니다.

이 방식은 토큰 제한을 안정적으로 만족시키는 데 유리하지만,
분할 과정에서 의미적으로 완전하지 않은 문장 단위로 잘릴 수 있다는 단점이 있습니다.

단순한 구조의 문서나,
문단 구성이 명확한 텍스트의 경우에는 CharacterTextSplitter로도 충분합니다.

하지만 실제 서비스 환경에서는
문서 길이와 구조가 제각각인 경우가 많기 때문에,
대부분의 RAG 시스템에서는 RecursiveCharacterTextSplitter를 기본 선택지로 사용합니다.

이는 Chunk 크기를 안정적으로 제어하면서도
검색 실패를 줄이는 데 유리하기 때문입니다.

Cell 028

```
with open("/content/state_of_the_union.txt") as f:  
    text = f.read()
```

Cell 029

```
#len      chunk size      ?      size      .  
#chunk_overlap chunk      chunk      size      .  
text_splitter = CharacterTextSplitter(separator="\n\n", chunk_size=1000, chunk_overlap=100, length_function =  
    len,)  
chunks = text_splitter.split_text(text)
```

Cell 030

Chunk의 내용을 확인해보겠습니다

Cell 031

```
print(chunks[0])
```

Captured output

```
Madam Speaker, Madam Vice President, our First Lady and Second Gentleman. Members of Congress and the Cabinet.  
Justices of the Supreme Court. My fellow Americans.  
  
Last year COVID-19 kept us apart. This year we are finally together again.  
  
Tonight, we meet as Democrats Republicans and Independents. But most importantly as Americans.  
  
With a duty to one another to the American people to the Constitution.  
  
And with an unwavering resolve that freedom will always triumph over tyranny.  
  
Six days ago, Russia's Vladimir Putin sought to shake the foundations of the free world thinking he could make  
it bend to his menacing ways. But he badly miscalculated.  
  
He thought he could roll into Ukraine and the world would roll over. Instead he met a wall of strength he never  
imagined.  
  
He met the Ukrainian people.  
  
From President Zelenskyy to every Ukrainian, their fearlessness, their courage, their determination, inspires  
the world.
```

Cell 032

각 chunk의 길이를 확인해보겠습니다,

Cell 033

```
length = []
for chunk in chunks:
    length.append(len(chunk))

print(length)
```

Captured output

```
[939, 995, 810, 962, 994, 883, 957, 952, 928, 915, 993, 702, 900, 950, 957, 958, 967, 996, 796, 866, 888, 966,
964, 977, 998, 948, 925, 924, 989, 965, 938, 936, 981, 965, 771, 982, 972, 977, 984, 999, 968, 801]
```

Cell 034

토큰 단위로 텍스트 분할해보기

LLM은 문장을 단어가 아닌 토큰(token) 단위로 처리합니다.
따라서 사람이 인식하는 단어 길이나 문자 수는
실제 모델이 처리하는 입력 길이와 정확히 일치하지 않을 수 있습니다.

이로 인해 문자 수나 단어 수를 기준으로 텍스트를 분할할 경우,
모델의 입력 토큰 제한을 초과하거나
예상보다 훨씬 짧은 문맥만 전달되는 문제가 발생할 수 있습니다.

실제 서비스 환경에서는 이러한 문제를 방지하기 위해,
토큰 단위를 기준으로 텍스트를 분할하는 방식을 사용합니다.
이제 토큰 기준으로 텍스트를 분할해보겠습니다.

Cell 035

```
!pip install tiktoken
```

Captured output

```
Requirement already satisfied: tiktoken in /usr/local/lib/python3.12/dist-packages (0.12.0)
Requirement already satisfied: regex>=2022.1.18 in /usr/local/lib/python3.12/dist-packages (from tiktoken)
(2025.11.3)
Requirement already satisfied: requests>=2.26.0 in /usr/local/lib/python3.12/dist-packages (from tiktoken)
(2.32.5)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from
requests>=2.26.0->tiktoken) (3.4.5)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from
requests>=2.26.0->tiktoken) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from
requests>=2.26.0->tiktoken) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from
requests>=2.26.0->tiktoken) (2026.2.25)
```

Cell 036

```
import tiktoken
tokenizer = tiktoken.get_encoding("cl100k_base")

def tiktoken_len(text):
    tokens = tokenizer.encode(
        text
    )
    return len(tokens)
```

Cell 037

```
tiktoken_length = []
for chunk in chunks:
    tiktoken_length.append(tiktoken_len(chunk))

print(length)
print(tiktoken_length)
```

Captured output


```
[939, 995, 810, 962, 994, 883, 957, 952, 928, 915, 993, 702, 900, 950, 957, 958, 967, 996, 796, 866, 888, 966,
964, 977, 998, 948, 925, 924, 989, 965, 938, 936, 981, 965, 771, 982, 972, 977, 984, 999, 968, 801]
[197, 198, 163, 190, 203, 182, 195, 197, 206, 205, 218, 148, 188, 205, 216, 215, 209, 224, 176, 187, 201, 197,
201, 215, 222, 202, 203, 204, 229, 206, 184, 204, 197, 194, 156, 200, 194, 221, 203, 225, 209, 187]
```

Cell 038

글자 수와 토큰 수의 차이를 확인할 수 있습니다 !

Cell 039

Step3 : TextEmbedding 사용해보기

Embedding은 텍스트를 컴퓨터가 계산할 수 있는 수치 벡터(vector) 형태로 변환하는 과정입니다.
이 벡터는 문장의 표면적인 형태가 아니라, 의미적 유사성을 반영하도록 설계되어 있습니다.

변환된 벡터는
VectorStore에 저장되거나,
새로운 질의(Query) 벡터와의 유사도 계산을 통해
의미적으로 가까운 문서를 검색하는 데 사용됩니다.

이러한 변환은 대규모 말뭉치로 사전 학습된
Embedding 전용 모델을 통해 이루어지며,
RAG 시스템에서 Retrieval 성능을 결정하는 핵심 요소입니다.

이번 실습에서는
OpenAI 임베딩 모델을 사용해
텍스트를 벡터로 변환해보겠습니다.

Cell 040

```
import openai
```

Cell 041

genai 라이브러리의 list_models 함수를 사용하여 사용 가능한 모델들의 목록을 가져옵니다.

Cell 042

```
client = openai.OpenAI()
```

Cell 043

```
for model in client.models.list():
    if "embedding" in model.id:
        print(model.id)
```

Captured output

```
text-embedding-3-small
text-embedding-3-large
text-embedding-ada-002
```

Cell 044

text-embedding-3-small은 가성비가 좋고, text-embedding-3-large는 성능이 더 강력합니다.

Cell 045

```
from langchain_openai import OpenAIEmbeddings

embedding_model = OpenAIEmbeddings(model="text-embedding-3-small")
```

Cell 046

만약 여러분들이 Gemini를 사용하여 구축중이시라면, embedding은 지역에 따라 사용이 제한됩니다. 주로 유럽권에서 제한되기 때문에, 다음 에러를 확인하신다면 Colab 파일의 서버 저장 위치를 확인 후, 다른 임베딩 모델로 변경해야합니다.

Error embedding content: 400 User location is not supported for the API use.

Cell 047

```
#!curl ipinfo.io
```

Cell 048

```
# 400 User location is not supported for the API use ,

# ! pip install -q sentence_transformers

#from langchain.embeddings import HuggingFaceEmbeddings
#embedding_model = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
```

Cell 049

embedding model 변수에 OpenAI 임베딩모델 혹은 huggingface의 임베딩모델이 할당되었을 것입니다. embed_documents 멤버 함수를 사용하여 새 문장을 변환해보겠습니다

Cell 050

```
embeddings = embedding_model.embed_documents(
    [
        "This is red apple.",
        "This is yellow banana.",
        "This is green lime.",
    ]
)
```

Cell 051

임베딩으로 잘 변환되었는지 확인해보겠습니다

Cell 052

```
print(embeddings[1])
```

Captured output

[Large captured output omitted for readability; the original notebook cell is retained.]

Cell 053

```
len(embeddings[1])
```

Captured output

1536

Cell 054

새로운 쿼리를 넣어, 임베딩끼리 유사도를 계산해보겠습니다

Cell 055

```
import numpy as np
from numpy import dot
from numpy.linalg import norm
def cos_sim(A, B):
    return dot(A, B)/(norm(A)*norm(B))
```

Cell 056

```
query = ["this is red fruit"]
```

Cell 057

```
e_query = embedding_model.embed_documents(query)
print(cos_sim(embeddings[0], e_query[0]))
print(cos_sim(embeddings[1], e_query[0]))
print(cos_sim(embeddings[2], e_query[0]))
```

Captured output

```
0.7477942151308524
0.48995458116791124
0.4084112226829356
```

Cell 058

빨간 사과와 빨간 과일의 유사도가 많이 높게 나왔습니다!

임베딩 모델은 사용 언어나 필요에 따라 다양하게 교체하여 사용할 수 있습니다.

해당 링크에서 여러 목록을 확인하실 수 있습니다.

https://python.langchain.com/docs/integrations/text_embedding/

Cell 059

Step4 : VectorStore 사용해보기

VectorStore는 텍스트를 Embedding 모델을 통해 벡터(vector)로 변환한 뒤, 이를 저장하고 관리하는 저장소입니다.

이 저장소는 단순한 데이터 보관 공간이 아니라,

벡터 간의 유사도를 빠르게 계산하고 탐색하기 위한 인덱싱 구조를 함께 포함하고 있습니다.

문서나 쿼리가 Embedding된 이후에는,

VectorStore를 통해 의미적으로 유사한 벡터를 효율적으로 검색할 수 있으며,

이 과정이 RAG 시스템의 Retrieval 단계를 담당하게 됩니다.

대표적인 VectorStore로는

Chroma, FAISS 등이 있으며,

각각 로컬 환경과 대규모 서비스 환경에서 널리 사용됩니다.

이번 실습에서는

구성이 단순하고 로컬 환경에서 바로 사용할 수 있는

ChromaDB를 사용해 VectorStore를 구성해보겠습니다.

Cell 060

```
!pip install chromadb
```

Captured output

```
[Large captured output omitted for readability; the original notebook cell is retained.]
```

Cell 061

```
!pip install langchain-chroma
```

Captured output

[Large captured output omitted for readability; the original notebook cell is retained.]

Cell 062

```
from langchain_chroma import Chroma
```

Cell 063

```
!pip install --upgrade opentelemetry-api  
!pip install --upgrade opentelemetry-sdk
```

Captured output

```
Requirement already satisfied: opentelemetry-api in /usr/local/lib/python3.12/dist-packages (1.40.0)  
Requirement already satisfied: importlib-metadata<8.8.0,>=6.0 in /usr/local/lib/python3.12/dist-packages (from  
  opentelemetry-api) (8.7.1)  
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from  
  opentelemetry-api) (4.15.0)  
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.12/dist-packages (from importlib-  
  metadata<8.8.0,>=6.0->opentelemetry-api) (3.23.0)  
Requirement already satisfied: opentelemetry-sdk in /usr/local/lib/python3.12/dist-packages (1.40.0)  
Requirement already satisfied: opentelemetry-api==1.40.0 in /usr/local/lib/python3.12/dist-packages (from  
  opentelemetry-sdk) (1.40.0)  
Requirement already satisfied: opentelemetry-semantic-conventions==0.61b0 in /usr/local/lib/python3.12/dist-  
  packages (from opentelemetry-sdk) (0.61b0)  
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from  
  opentelemetry-sdk) (4.15.0)  
Requirement already satisfied: importlib-metadata<8.8.0,>=6.0 in /usr/local/lib/python3.12/dist-packages (from  
  opentelemetry-api==1.40.0->opentelemetry-sdk) (8.7.1)  
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.12/dist-packages (from importlib-  
  metadata<8.8.0,>=6.0->opentelemetry-api==1.40.0->opentelemetry-sdk) (3.23.0)
```

Cell 064

제일 처음에 사용했던, PDF를 다시 사용하도록 합니다!

Cell 065

```
#  
loader = PyPDFLoader("/content/Demian.pdf")  
pages = loader.load_and_split()  
  
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=0, length_function = tiktoken_len)  
docs = text_splitter.split_documents(pages)
```

Cell 066

```
#!pip show chromadb
```

Cell 067

Chroma에 임베딩 시킵니다

Cell 068

```
db = Chroma.from_documents(docs, embedding_model)
```

Cell 069

이제 쿼리를 날려보겠습니다

Cell 070

```
query = "how Demian look like?"  
docs = db.similarity_search(query)
```

Cell 071

```
print(docs[0].page_content)
```

Captured output

```
DEMIAN
with a feeling of nausea, I noticed Demian's expression.
He had not thrust himself to the front but stood right
at the back, looking elegant and at ease as usual. His
glance seemed directed at the horse's head, and again it
. showed that deep, quiet, almost fanatical yet passionate
absorption. I could not help staring at him for some
moments and it was then that I felt aware of a very
uncanny sensation in my remote consciousness. I saw
Demian's face and remarked that it was not a boy's face
but a man's and then I saw, or rather became aware, that
it was not really the face of a man either; it had some-
thing different about it, almost a feminine element. And
for the time being his face seemed neither masculine
nor childish, neither old nor young but a hundred years
old, almost timeless and bearing the mark of other
periods of history than our own. Animals might look
thus, trees or stars. I did not know then, of course, I
did not feel exactly what I am writing about it now, as
an adult, but it was something of the kind. Perhaps he
was handsome, perhaps I found him attractive, perhaps
he repelled me too, I could not even be sure of that. All
I saw was that he was different from the rest of us, that
he was like an animal, a spirit or an image. I cannot
describe him except to say that he was different, un-
imaginably different from the rest of us. My memory is a
blank about everything else and perhaps even what I
have described is to some extent gleaned from later
impressions.
Only when I was several years older did I come into
close touch with him. Contrary to the usual custom,
Demian had not been confirmed with the other boys of
56
Downloaded from https://www.holybooks.com
```

Cell 072

Face, features, looks like 등 데미안의 생김새를 담고 있는 페이지가 출력되었습니다
굉장히 빠른 속도로 검색했습니다!

Cell 073

Step5 : Retriever 사용해보기

Retriever는 사용자의 질문을 Embedding 모델을 통해 벡터로 변환한 뒤,
VectorStore에 저장된 문서 벡터들과 비교하여
의미적으로 가장 유사한 문서(Chunk)를 찾아 반환하는 역할을 합니다.

즉, Retriever는

RAG 시스템에서 “어떤 정보를 LLM에게 참고 자료로 줄 것인가”를 결정하는 핵심 컴포넌트이며,
검색 결과의 품질이 곧 최종 답변의 품질로 이어집니다.

Cell 074

```
!pip install -U langchain langchain-classic
```

Captured output

```
[Large captured output omitted for readability; the original notebook cell is retained.]
```

Cell 075

```
from langchain_classic.chains.retrieval_qa.base import RetrievalQA
```

Cell 076

긴 문서 전체를 한 번에 LLM에 전달하는 대신,
Retriever와 LLM을 결합한 RetrievalQA 체인을 사용하여
문서에서 질문과 관련된 부분만 검색하고,
그 결과를 바탕으로 답변을 생성합니다.

이를 통해 길이가 긴 문서에서도
토큰 제한을 넘지 않으면서, 근거 기반의 질의응답을 수행할 수 있습니다.

Cell 077

```
from langchain_core.callbacks.streaming_stdout import StreamingStdOutCallbackHandler

# OpenAI
# streaming=True callbacks
llm = ChatOpenAI(
    model="gpt-4o",          # "gpt-4o-mini"
    temperature=0.0,
    streaming=True,          #
    callbacks=[StreamingStdOutCallbackHandler()], #
)
```

Cell 078

체인의 종류와 검색(Retrieval) 방식,
그리고 그에 따른 주요 파라미터를 설정합니다.

이 단계에서는
Retriever가 어떤 전략으로 문서를 검색할지,
그리고 몇 개의 문서를 LLM에게 전달할지를 결정하게 됩니다.
이 선택은 최종 답변의 품질과 직접적으로 연결됩니다.

예를 들어,
MMR(Maximal Marginal Relevance) 방식은
쿼리와 유사도뿐만 아니라 문서 간의 중복을 줄이고 다양성을 확보하는 재정렬(Re-ranking) 전략입니다.

실무 환경에서는 단일 문서에 정보가 물리는 것을 방지하고,
LLM이 보다 풍부한 문맥을 참고하도록 하기 위해
MMR 방식이 자주 사용됩니다.

Cell 079

```
qa = RetrievalQA.from_chain_type(llm, chain_type="stuff",
                                  retriever=db.as_retriever(
                                      search_type="mmr",
                                      search_kwargs={"k": 3, "fetch_k": 10}),
                                  return_source_documents=True)
```

Cell 080

위 코드에서 짚고 넘어갈 파라미터는 다음과 같습니다
chain_type="stuff"

검색된 문서(Chunk)를 그대로 하나의 Prompt에 모두 삽입하는 방식입니다.
구조가 단순하고 이해하기 쉬워,
RAG 구조를 처음 학습하거나 프로토타입을 만들 때 적합합니다.
단점으로는 문서 수가 많아질 경우
토큰 사용량이 빠르게 증가할 수 있습니다.
실무에서는 초기 검증 단계에서는 stuff를,
문서 수가 많아지면 map_reduce나 refine 방식으로 확장합니다.

retriever

VectorStore에서 어떤 문서를 검색할지 결정하는 검색 모듈입니다.

검색 전략과 파라미터 설정에 따라 LLM이 참고하는 정보의 범위와 품질이 달라집니다.

```
search_type="mmr"
```

MMR(Maximal Marginal Relevance) 검색 방식을 사용합니다. 쿼리와 유사도뿐만 아니라, 문서 간 중복을 줄여 다양한 문맥을 확보하는 Re-ranking 전략입니다. 실무 환경에서 단일 문서 편향을 줄이기 위해 자주 사용됩니다.

```
search_kwargs={"k": 3, "fetch_k": 10}
```

- fetch_k

VectorStore에서 우선적으로 가져올 후보 문서 개수입니다. Re-ranking 이전 단계에서 사용됩니다.

- k

최종적으로 LLM에게 전달할 문서(Chunk)의 개수입니다.

일반적으로 fetch_k > k 로 설정하여 후보 풀을 넉넉히 확보한 뒤, 품질 좋은 문서만 선별하는 방식을 사용합니다.

```
return_source_documents=True
```

답변 생성에 사용된 원문 문서(Chunk)를 함께 반환합니다. 이를 통해 답변의 출처를 사용자에게 표시하거나 검색 품질을 디버깅하고 RAG 성능을 평가할 수 있습니다. 실무 서비스에서는 거의 필수적으로 사용하는 옵션입니다.

Cell 081

```
query = "how demian looks like"
result = qa(query)
```

Captured output

```
/tmp/ipykernel_12293/3336337621.py:2: LangChainDeprecationWarning: The method `Chain.__call__` was deprecated in
langchain-classic 0.1.0 and will be removed in 1.0. Use `invoke` instead.
result = qa(query)
```

```
Demian is described as having a face that is neither distinctly masculine nor childish, neither old nor young,
but rather timeless and bearing the mark of other periods of history. His face has an almost feminine
element and is different from the rest of the people around him. It is suggested that his appearance is
unique, almost like an animal, a spirit, or an image, making him unimaginably different from others.
```

Cell 082

마크다운 형식으로 출력해봅니다

Cell 083

```
from IPython.display import Markdown, display
display(Markdown(result["result"]))
```

Captured output

```
<IPython.core.display.Markdown object>
```

Cell 084

RAG를 사용하지 않은 llm 호출도 시도해보세요!

Cell 085

```
llm2 = ChatOpenAI(
    model="gpt-4o")
request = llm2.invoke("how demian looks like")
display(Markdown(request.content))
```

Captured output

<IPython.core.display.Markdown object>

Cell 086

Quiz

결과의 어떤 부분을 관찰하였을 때, RAG 시스템의 결과를 신뢰할 수 있겠다 생각하셨나요?

Cell 087

Answer

원문에서 답변의 출처를 확인할 수 있었습니다.

Cell 088

6. 완성 예제

앞에서 진행한 내용으로, Demian을 다시 한번 읽어봅시다!
완성하여 제출해주세요~

Cell 089

필요한 라이브러리를 모두 다운받습니다

Cell 090 | Completed implementation

```
%pip install -qU langchain langchain-openai langchain-community langchain-text-splitters langchain-chroma  
langchain-classic pypdf tiktoken
```

Cell 091

Text splitter 사용을 위한 준비입니다

Cell 092 | Completed implementation

```
import os  
from pathlib import Path  
import tiktoken  
from langchain_community.document_loaders import PyPDFLoader  
from langchain_text_splitters import RecursiveCharacterTextSplitter  
from langchain_openai import ChatOpenAI, OpenAIEmbeddings  
from langchain_chroma import Chroma  
from langchain_classic.chains.retrieval_qa.base import RetrievalQA  
from IPython.display import Markdown, display  
  
# Colab Secrets is used in the earlier setup cell; this fallback keeps the  
# completed section safe to rerun when the key is already in the environment.  
if not os.environ.get("OPENAI_API_KEY"):  
    try:  
        from google.colab import userdata  
        key = userdata.get("OPENAI_API_KEY")  
        if key:  
            os.environ["OPENAI_API_KEY"] = key  
    except (ImportError, AttributeError, KeyError):  
        pass  
  
tokenizer = tiktoken.get_encoding("cl100k_base")  
  
def tiktoken_len(text: str) -> int:  
    return len(tokenizer.encode(text))
```


Step 1 Document loader

Cell 094 | Completed implementation

```
# The exercise uses Hermann Hesse's Demian as the source document.
pdf_candidates = [
    Path("/content/Demian.pdf"),
    Path("Demian.pdf"),
    Path("/content/drive/MyDrive/Demian.pdf"),
]
pdf_path = next((path for path in pdf_candidates if path.exists()), None)
if pdf_path is None:
    raise FileNotFoundError(
        "Demian.pdf was not found. Upload it to Colab or place it in the notebook folder."
    )

loader = PyPDFLoader(str(pdf_path))
pages = loader.load()
pages = [page for page in pages if page.page_content.strip()]
assert pages, "The source PDF contains no extractable text."
print(f"Loaded {len(pages)} pages from {pdf_path.name}.")
```

Step 2 Text splitters

Cell 096 | Completed implementation

```
# Split pages into overlapping, token-aware chunks for retrieval.
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    length_function=tiktoken_len,
    separators=["\n\n", "\n", " ", ""],
)
docs = text_splitter.split_documents(pages)
assert docs, "No chunks were produced from the source document."
print(f"Created {len(docs)} chunks; first chunk has {tiktoken_len(docs[0].page_content)} tokens.")
```

Step 3 Vector Embeddings

Cell 098 | Completed implementation

```
# OpenAI embeddings map chunks and queries into the same vector space.
embedding_model = OpenAIEmbeddings(model="text-embedding-3-small")
```

Cell 099 | Completed implementation

```
# Chroma stores the chunk vectors and their source metadata.
db = Chroma.from_documents(
    documents=docs,
    embedding=embedding_model,
    collection_name="demian_rag",
)
print("Vector store is ready.")
```

Step 4 Retrievers

Cell 101 | Completed implementation

```
# MMR returns relevant chunks while reducing near-duplicate results.
retriever = db.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 3, "fetch_k": 10},
)
```

Cell 102 | Completed implementation

```
# RetrievalQA connects the retriever to the answer-generating LLM.
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
qa = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True,
)
```

Cell 103

Step 5 Question Answering

Cell 104 | Completed implementation

```
query = "How is Demian's appearance described?"
result = qa.invoke({"query": query})
answer = result["result"]
print(answer)
```

Cell 105 | Completed implementation

```
display(Markdown(answer))
print(f"Retrieved source chunks: {len(result.get('source_documents', []))}")
for index, document in enumerate(result.get("source_documents", []), start=1):
    metadata = document.metadata
    page = metadata.get("page_label", metadata.get("page", "?"))
    print(f"[{index}] source page: {page}")
```